

Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Corso di preparazione per la selezione territoriale delle Olimpiadi di Informatica

Lezione 5. Programmazione Dinamica

Giuseppe F. Italiano

7 aprile 2022

LUISS

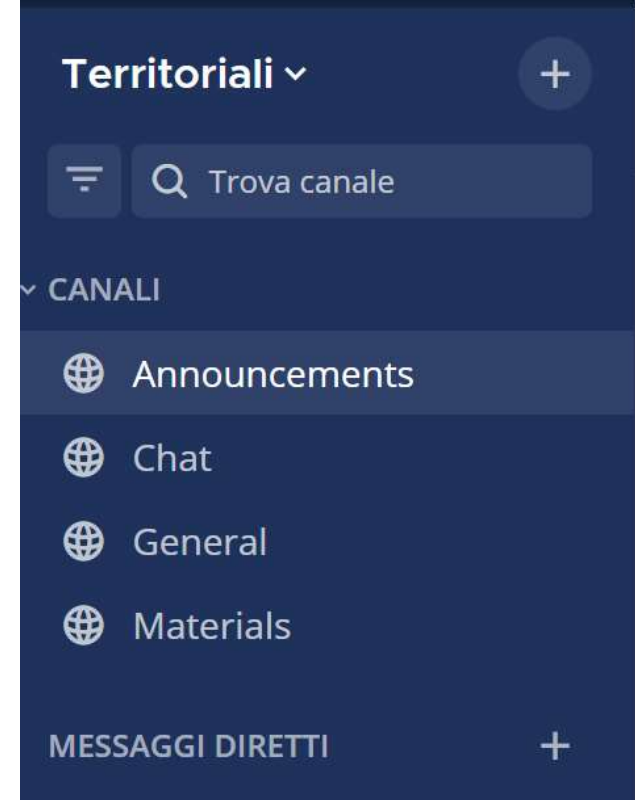


Reminder: LuissTrain

<https://coding.luiss.ml/train/territoriali/>

- 1. *Announcements*** (e.g., link video lezioni)
- 2. *Chat*** (fai qui le tue domande durante le lezioni)
- 3. *General*** (discussioni interesse generale)
- 4. *Materials*** (materiale didattico, slide prima delle lezioni, etc...)

- Controlla i canali: spesso troverai qui le risposte alle tue domande!
- Non inquinare i canali! Contribuisci a mantenerli puliti e non postare contenuti impropri. Grazie!



Lezioni

Puoi accedere alle lezioni tramite YouTube o Webex:

 **YouTube** (preferibile se hai problemi di connessione)

 Cisco **webex** (puoi diventare ***Chad (Stacy) per un giorno*** condividendo la tua soluzione ai problemi che consideriamo di volta in volta! Windows, MacOS)

Link pubblicati ***solo*** su Announcement (LuisTrain)



Contest



Giga Chad



	raggio	spia	microfono	mutanti	Total
eric169	25	25	25	4.5	79.5
leodra	25	5.5		24.75	55.25

Mega Chad / Stacy

Punteggio compreso tra 40 e 50.75



Chad / Stacy

Punteggio compreso tra 30 e 40



Povero Gabbiano

Punteggio compreso tra 10 e 20



Rage Quittato?

Punteggio inferiore a 10



Soluzioni del Contest



Dato che abbiamo visto le soluzioni al contest...

- Oggi correggeremo insieme solo qualcuno (2?) dei compiti a casa.
- E il resto? Siete bravissimi, e sapete farli da soli.
- Potete sempre discuterne le soluzioni sulla Chat di LuissTrain (e anche su questo siete bravissimi!)
- Meglio dedicare il tempo a disposizione per imparare cose nuove?

Compiti a casa

- Domino massimale: <https://training.olinfo.it/#/task/domino/statement>
- Mappa antica: <https://training.olinfo.it/#/task/mappa/statement>
- Lotteria di quadri: https://training.olinfo.it/#/task/abc_quadri/statement
- Sorting proximity: https://training.olinfo.it/#/task/ois_proximity/statement
- Paletta Sort (con divide et impera):
<https://training.olinfo.it/#/task/paletta/statement>
- Missioni segrete (con ricorsione):
<https://training.olinfo.it/#/task/missioni/statement>

Programma di oggi (ambizioso?)

1. Programmazione Dinamica
2. Un problema dalle Territoriali (2004): ***Poldo*** 
3. Un problema dalle Territoriali (2016): ***discesa*** 
4. Un problema da LUISSMatics (2018): ***laurea*** 
5. Un problema dalle pre-OII (2020): ***sushi*** 
6. Un problema delle pre-OII (2018): ***treni*** 
7. Esercizi Olimpiadi (compiti da fare a casa per la prossima volta)

Programmazione Dinamica







Programmazione Dinamica

(visita guidata molto semplificata)



Numeri di Fibonacci

- Definiti come relazione di ricorrenza:

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{se } n \geq 3 \\ 1 & \text{se } n = 1, 2 \end{cases}$$

- Problema: come calcolare F_n ?

X: Non puoi mettere la sequenza di Fibonacci su immagini a caso

Prof. Italiano:



Algoritmo recursive-fibonacci

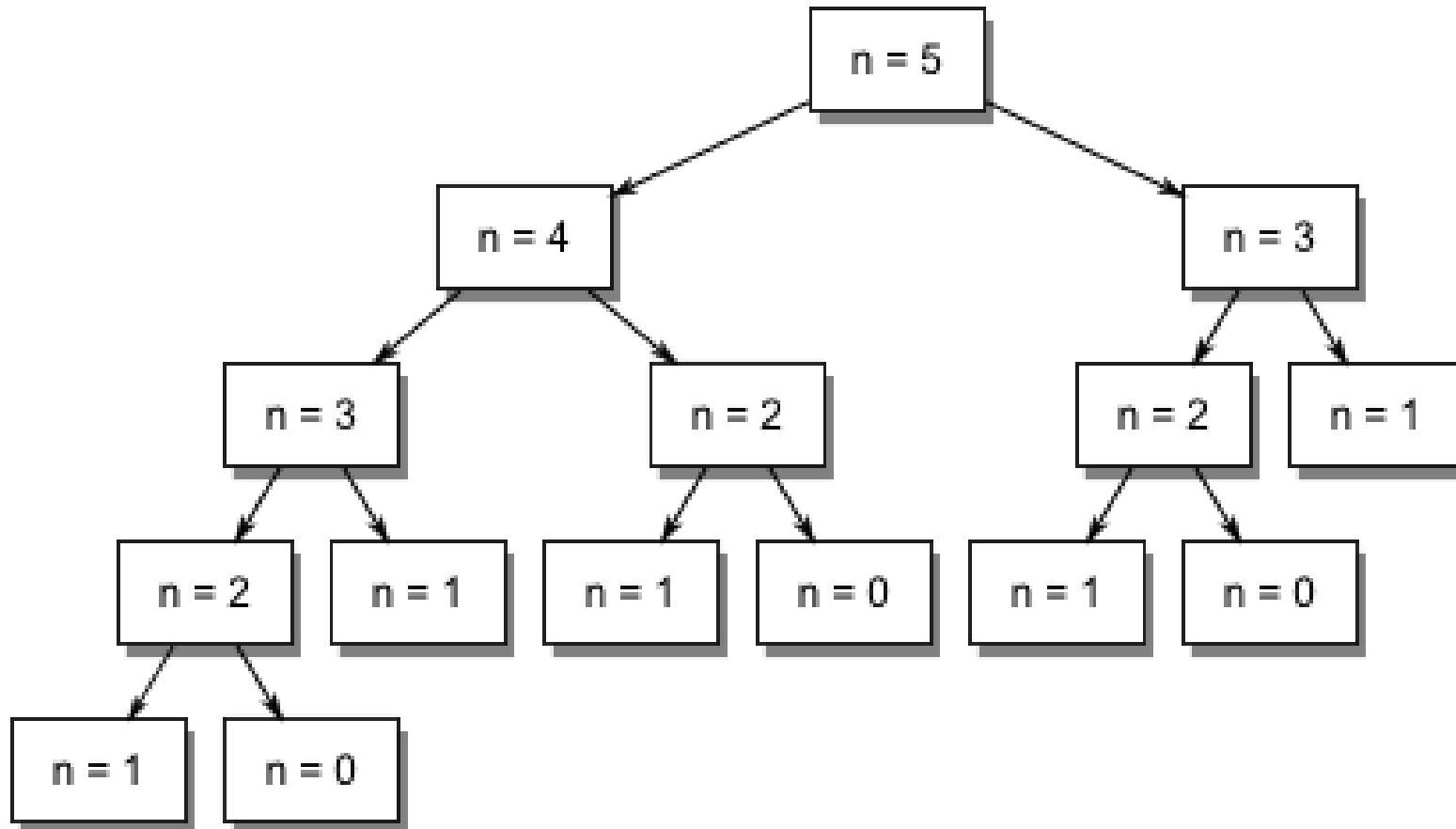
Sembra naturale utilizzare direttamente la definizione (ricorsiva):

```
algoritmo fibonacci(intero  $n$ )  $\rightarrow$  intero  
  if ( $n \leq 2$ ) then return 1  
  else return fibonacci( $n-1$ ) + fibonacci( $n-2$ )
```

E' una soluzione accettabile?

 **recursive-fibonacci.cc**

Algoritmo recursive-fibonacci



Possiamo fare di meglio?

Approccio (doppiamente) ricorsivo produce un algoritmo molto lento:

$$T(n) \approx O(F_n) \approx O(2^n)$$

Perché è lento?

Continua a ricalcolare più volte la soluzione dello stesso sottoproblema! (Alzheimer???)

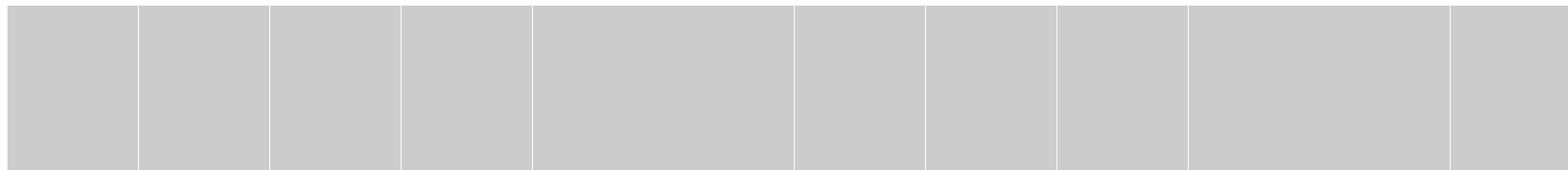
Programmazione Dinamica

Per evitare di risolvere più volte lo stesso sottoproblema, sembra naturale memorizzare la soluzione dei vari sottoproblemi in un array.

Memorizza F_i in $\text{Fib}[i]$

Per calcolare F_i ($\text{Fib}[i]$) possiamo utilizzare le soluzioni memorizzate in $\text{Fib}[i-1]$ e $\text{Fib}[i-2]$

1 1 2 3 5 8 13 21 34 55 89 144



1 2 3 4 5 6 7 8 9 10 11 12

Algoritmo array-fibonacci

Algoritmo array-fibonacci

```
algoritmo fibonacci(intero n)  $\rightarrow$  intero  
  sia Fib un array di n interi  
   $Fib[1] \leftarrow Fib[2] \leftarrow 1$   
  for  $i = 3$  to  $n$  do  
     $Fib[i] \leftarrow Fib[i-1] + Fib[i-2]$   
  return  $Fib[n]$ 
```

 [array-fibonacci.cc](#)

Possiamo semplificare ulteriormente?

- Ci serve proprio una tabella così grande (array di dimensione n)?
- Non servono tutti i valori di F_n precedenti, ma solo gli ultimi due: bastano tre variabili!

algoritmo fibonacci(*intero* n) $\rightarrow$ *intero*

$a \leftarrow b \leftarrow 1$

for $i = 3$ **to** n **do**

$c \leftarrow a + b$

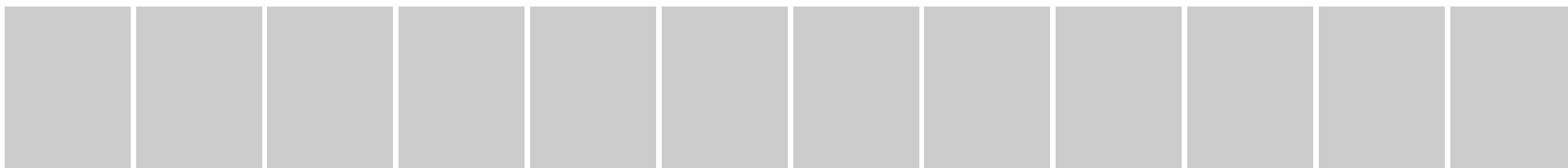
$a \leftarrow b$

$b \leftarrow c$

return b

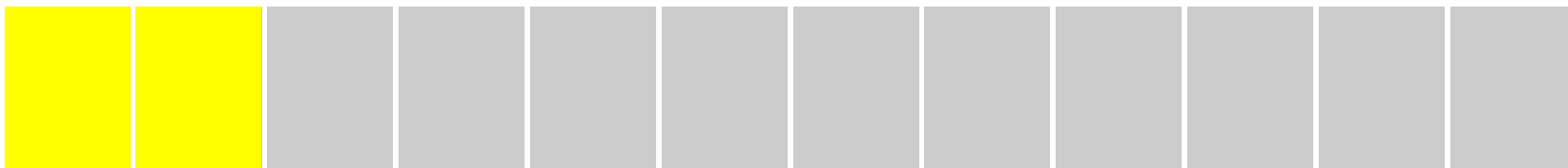
Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)



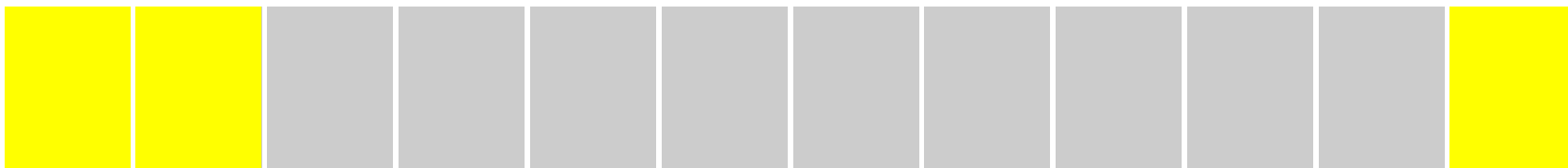
Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella



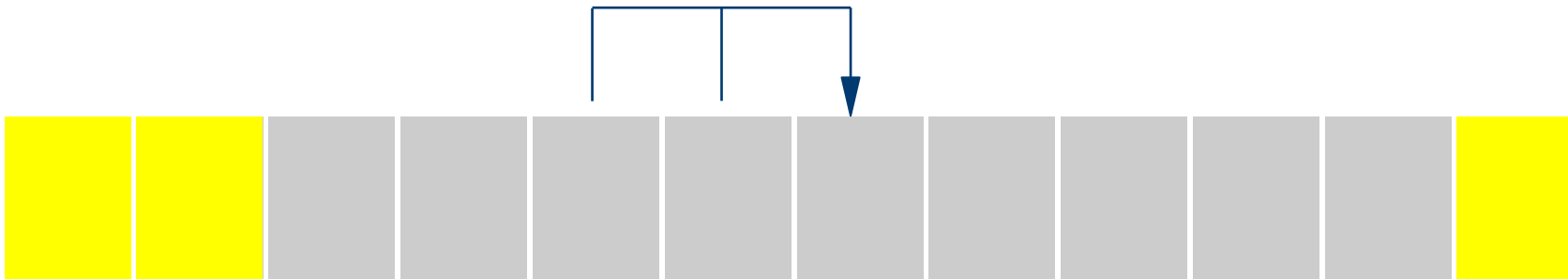
Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?



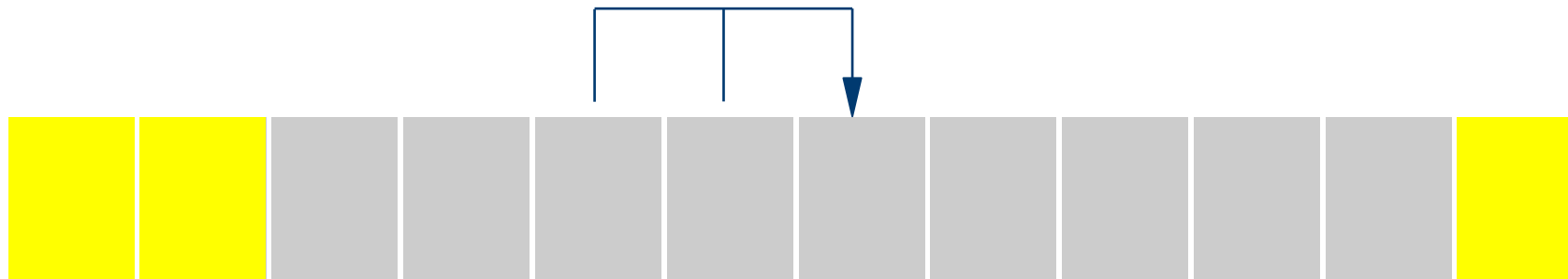
Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?
4. Riempire la tabella (da condizioni iniziali a soluzione)



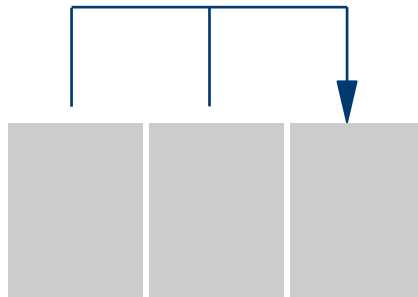
Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?
4. Riempire la tabella (da condizioni iniziali a soluzione)
5. Verificare cosa ci serve effettivamente della tabella (ridurne le dimensioni?)



Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?
4. Riempire la tabella (da condizioni iniziali a soluzione)
5. Verificare cosa ci serve effettivamente della tabella (ridurne le dimensioni?)



Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?
4. Riempire la tabella (da condizioni iniziali a soluzione)
5. Verificare cosa ci serve effettivamente della tabella (ridurne le dimensioni?)

Le 5 Leggi della Programmazione Dinamica

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?
4. Riempire la tabella (da condizioni iniziali a soluzione)
5. Verificare cosa ci serve effettivamente della tabella (ridurne le dimensioni?)

Punto critico

1. Definire opportunamente sottoproblemi (tabella)
2. Condizioni iniziali della tabella
3. Dove sta la soluzione?
4. Riempire la tabella (da condizioni iniziali a soluzione)
5. Verificare cosa ci serve effettivamente della tabella (ridurne le dimensioni?)

Definizione dei sottoproblemi è il passo più critico

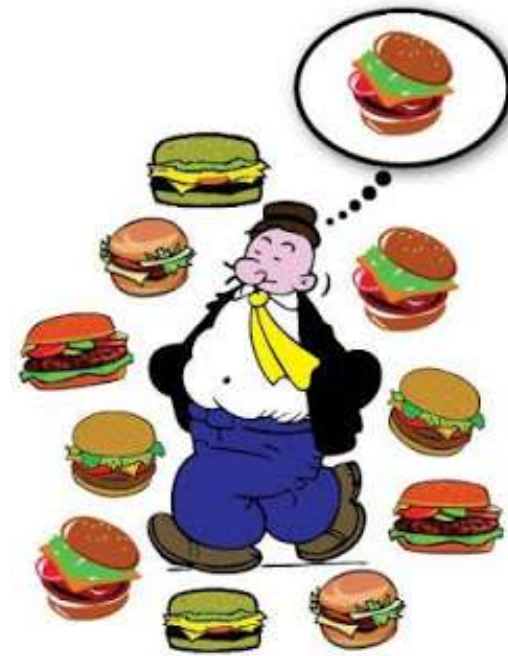
Fatta questa scelta, tutto il resto viene di conseguenza!





Un problema dalle Territoriali (2004): Poldo

<https://training.olinfo.it/#/task/poldo/statement>



La dieta di Poldo (Poldo)

Descrizione del problema

Il dottore ordina a Poldo di seguire una dieta. Ad ogni pasto non può mai mangiare un panino che abbia un peso maggiore o uguale a quello appena mangiato. Quando Poldo passeggia per la via del suo paese, da ogni ristorante esce un cameriere proponendo il menù del giorno. Ciascun menù è composto da una serie di panini, che verranno serviti in un ordine ben definito, e dal peso di ciascun panino. Poldo, per non violare la regola della sua dieta, una volta scelto un menù, può decidere di mangiare o rifiutare un panino; se lo rifiuta il cameriere gli servirà il successivo e quello rifiutato non gli sarà più servito.

Si deve scrivere un programma che permetta a Poldo, leggendo un menù, di capire qual è il numero massimo di panini che può mangiare per quel menù senza violare la regola della sua dieta.

Riassumendo, Poldo può mangiare un panino se e solo se soddisfa una delle due condizioni:

- il panino è il primo che mangia in un determinato pasto;
- il panino non ha un peso maggiore o uguale all'ultimo panino che ha mangiato in un determinato pasto.

Dati di input

La prima linea del file `input.txt` contiene il numero m di panini proposti nel menù. Le successive m linee contengono un numero intero non negativo che rappresenta il peso del panino che verrà servito. I panini verranno serviti nell'ordine in cui compaiono nell'input.

Dati di output

Il file `output.txt` contiene il massimo numero di panini che Poldo può mangiare rispettando la dieta.

Assunzioni

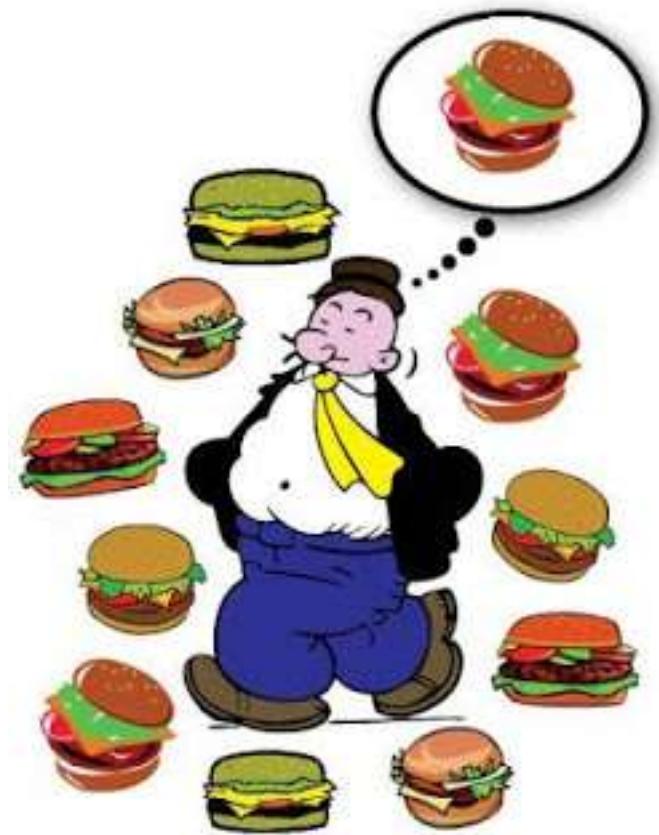
- I pesi dei panini sono espressi in grammi, e un panino pesa al massimo 10.000 grammi.
- $1 \leq m \leq 10.000$

Esempi di input/output

File input.txt	File output.txt
8 389 207 155 300 299 170 158 65	6
File input.txt	File output.txt
3 22 23 27	1

File input.txt	File output.txt
22 15 14 15 389 201 405 204 130 12 50 13 26 190 305 25 409 3011 43 909 987 1002 900	6

Poldo: Soluzione



Greedy?

Greedy?

Mangio il primo panino

Da quel punto in poi, mangio ogni panino che posso prendere

Greedy?

Mangio il primo panino

Da quel punto in poi, mangio ogni panino che posso prendere

0	1	2	3	4	5	6	7	8	9
100	200	190	180	170	160	150	140	130	120

Ricorsione?



Risoluzione: Programmazione Dinamica

0	1	2	3	4	5	6	7	8	9
8	38	20	15	30	29	18	16	9	10

1. Qual è il sottoproblema da risolvere?

Come succede di solito (anche con la programmazione dinamica), ci sono varie possibilità...

Risoluzione: Programmazione Dinamica

0	1	2	3	4	5	6	7	8	9
8	38	20	15	30	29	18	16	9	10

Approccio 1

Più intuitivo, approccio più o meno standard

Produrrà soluzione $O(n^2)$

Tutto sommato, molto semplice

Non è la soluzione più veloce. Ce la farà a passare tutti i test? OK per le territoriali

Solo i Giga Chad riescono a trovare una soluzione $O(n \log n)$



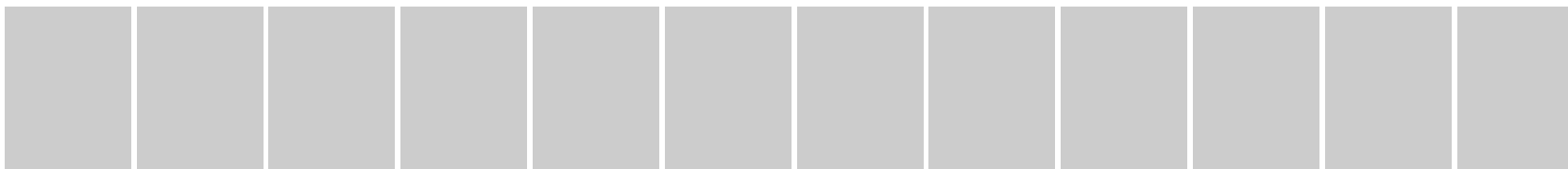
Risoluzione: Programmazione Dinamica

0	1	2	3	4	5	6	7	8	9
8	38	20	15	30	29	18	16	9	10



1. Qual è il sottoproblema da risolvere?

Approccio 1: Quanti panini può mangiare al massimo Poldo nell'intervallo $[1, i]$.



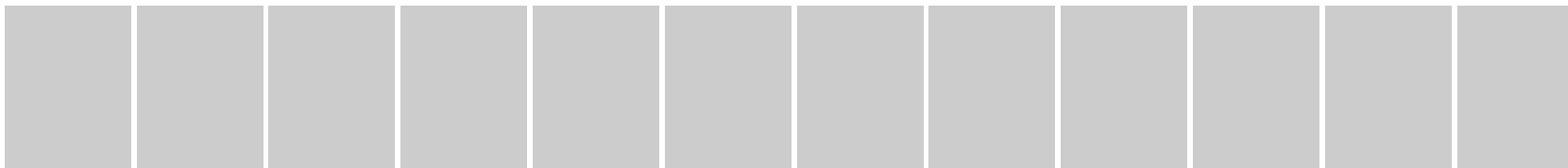
Risoluzione: Programmazione Dinamica

0	1	2	3	4	5	6	7	8	9
8	38	20	15	30	29	18	16	9	10



1. Qual è il sottoproblema da risolvere?

Approccio 1: Quanti panini può mangiare al massimo Poldo nell'intervallo $[1, i]$ se decide di mangiare il panino i .



Risoluzione: Programmazione Dinamica

0	1	2	3	4	5	6	7	8	9
8	38	20	15	30	29	18	16	9	10



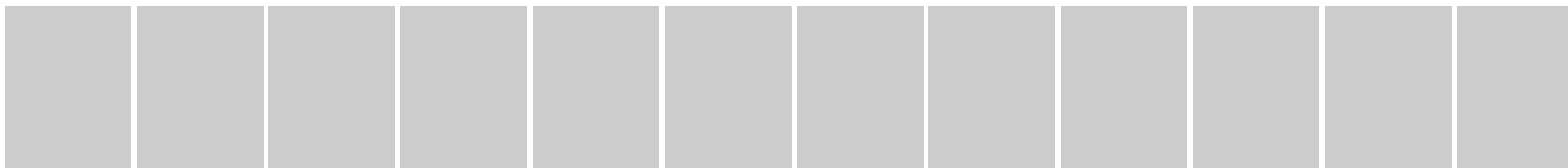
1. Qual è il sottoproblema da risolvere?

Approccio 1: Quanti panini può mangiare al massimo Poldo nell'intervallo $[1, i]$ se decide di mangiare il panino i .

La tabella quindi sarà semplicemente un array *quantiPanini[]*.

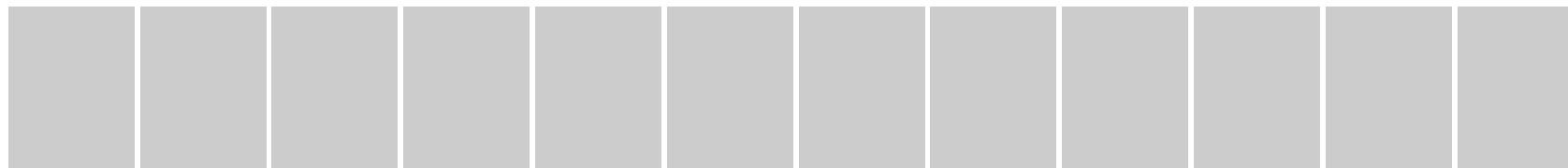
- Darà luogo a una soluzione quadratica standard ($O(n^2)$)
- Scriveremo codice che si basa su questo approccio

Lasciamo sottoproblema che porta a $O(n \log n)$ ai Giga Chad.



Risoluzione: Programmazione Dinamica

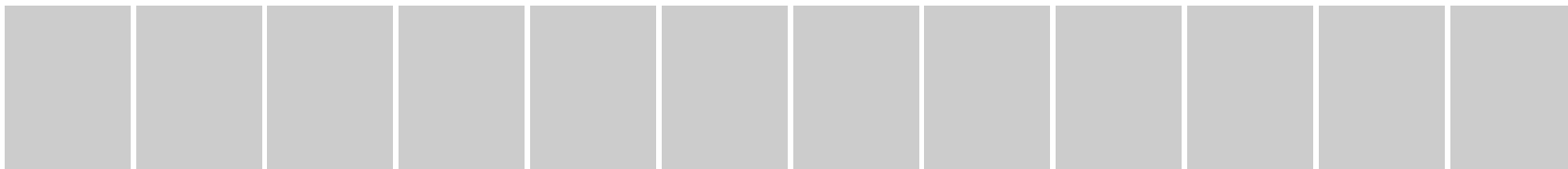
2. Condizioni iniziali?



Risoluzione: Programmazione Dinamica

2. Condizioni iniziali?

Sottoproblema per la prima posizione, la cui soluzione è 1: Se ho solo un panino a disposizione, lo mangio. *quantiPanini[0]=1*

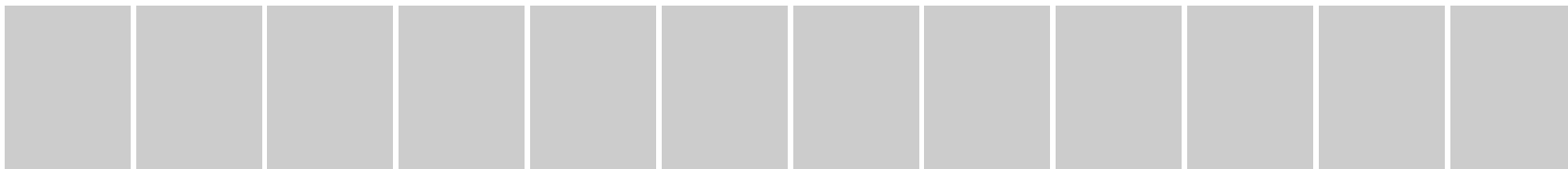


Risoluzione: Programmazione Dinamica

2. Condizioni iniziali?

Sottoproblema per la prima posizione, la cui soluzione è 1: Se ho solo un panino a disposizione, lo mangio. *quantiPanini[0]=1*

3. Dove si trova la soluzione?



Risoluzione: Programmazione Dinamica

2. Condizioni iniziali?

Sottoproblema per la prima posizione, la cui soluzione è 1: Se ho solo un panino a disposizione, lo mangio. $quantiPanini[0]=1$

3. Dove si trova la soluzione?

L'output del problema sarà il massimo tra tutti i sottoproblemi:

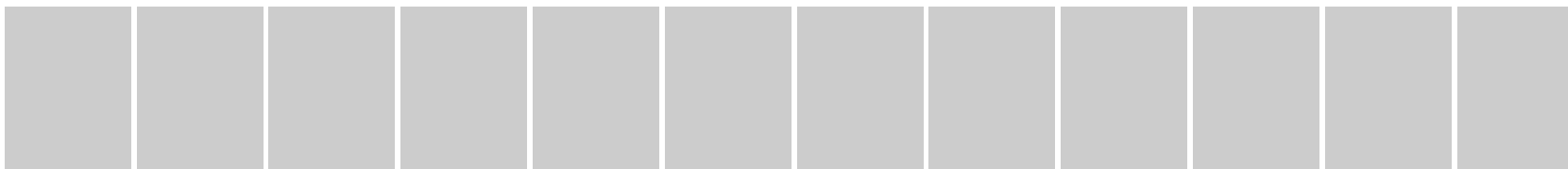
$$\max_i \{ quantiPanini[i] \}$$

Nota che la soluzione non è memorizzata in un punto specifico della tabella, ma la devo andare a cercare!!!



Risoluzione: Programmazione Dinamica

4. Come riempiamo la tabella?

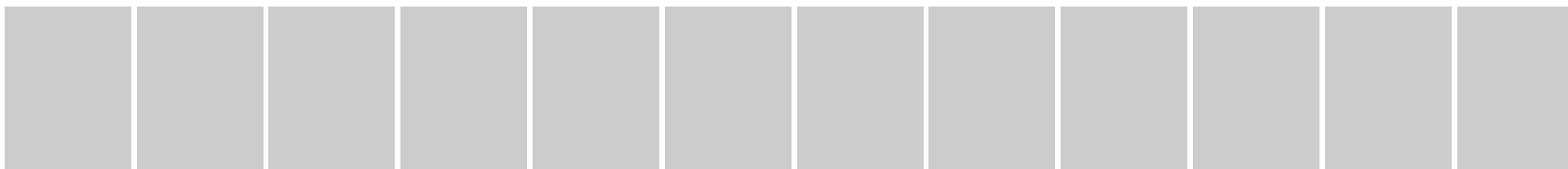




Risoluzione: Programmazione Dinamica

4. Come riempiamo la tabella?

Per trovare il valore nella posizione i (quanti panini posso mangiare al massimo se mangio il panino i), esamino qualsiasi $j < i$ e calcolo quanti panini posso mangiare se mangio il panino j e subito dopo il panino i (ammesso che ciò sia possibile).

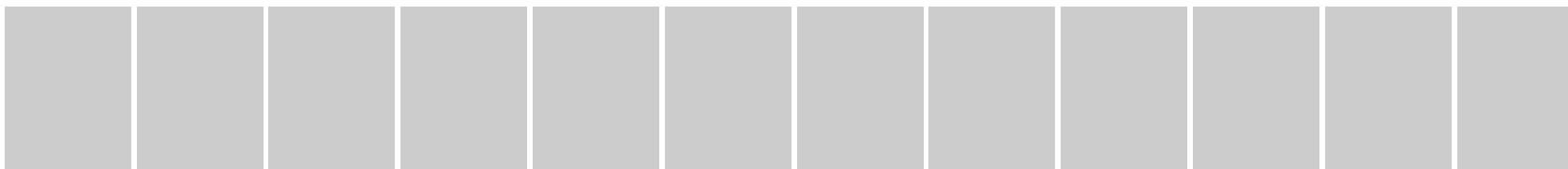




Risoluzione: Programmazione Dinamica

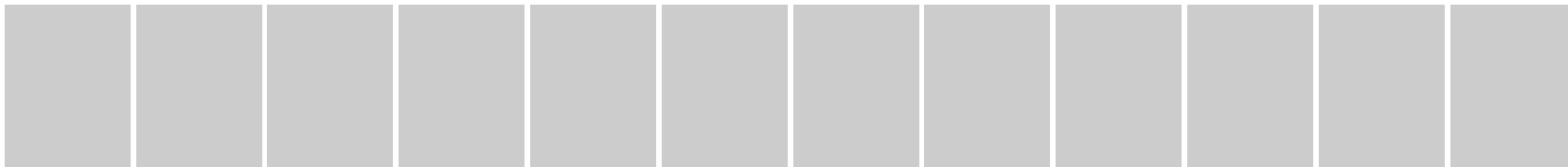
0	1	2	3	4	5	6	7	8	9
8	38	20	15	30	29	18	16	9	10

quantiPanini[5] può essere *quantiPanini[1] + 1* oppure *quantiPanini[4] + 1*



Risoluzione: Programmazione Dinamica

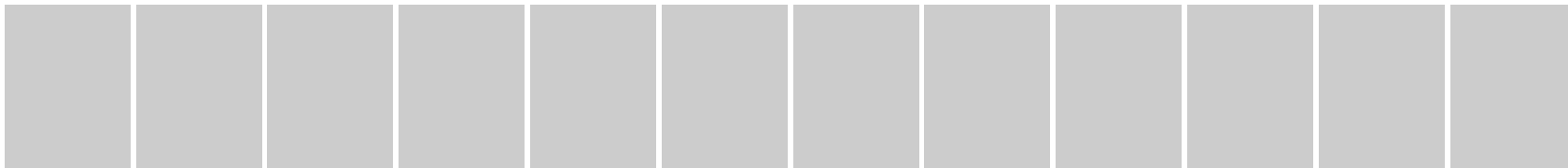
5. Possiamo eliminare qualcosa dalla tabella?



Risoluzione: Programmazione Dinamica

5. Possiamo eliminare qualcosa dalla tabella?

Non sembra molto semplice.



Implementazione

Un problema dalle Territoriali (2016): Discesa massima

<https://training.olinfo.it/#/task/discesa/statement>

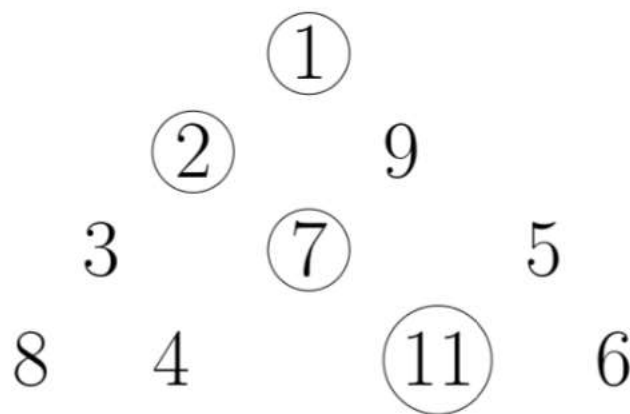


Discesa massima (discesa)

Difficoltà $D = 2$

Descrizione del problema

Come ben sanno gli studenti che hanno passato le selezioni scolastiche delle Olimpiadi di Informatica di quest'anno, data una piramide di numeri, definiamo una **discesa** come *una sequenza di numeri ottenuti partendo dalla cima della piramide e passando per uno dei due numeri sottostanti, fino a giungere alla base della piramide*. Inoltre, il **valore** di una discesa è definito come la somma dei numeri della discesa. La **discesa massima** di una piramide è quella che ha il massimo valore tra tutte le discese della piramide.



Valore della
discesa
evidenziata:
 $1+2+7+11=21$

Valore della
discesa
massima:
 $1+9+7+11=28$

Dati di input

Il file input.txt è composto da $1 + A$ righe di testo. La prima riga contiene A , un intero positivo rappresentante l'altezza della piramide. Le seguenti A righe descrivono effettivamente la piramide: l' i -esima riga (con i compreso tra 1 e A) contiene i interi positivi rappresentanti l' i -esimo "livello" della piramide.

Dati di output

Il file output.txt è composto da una sola riga contenente un intero positivo: il valore della discesa massima.

Assunzioni

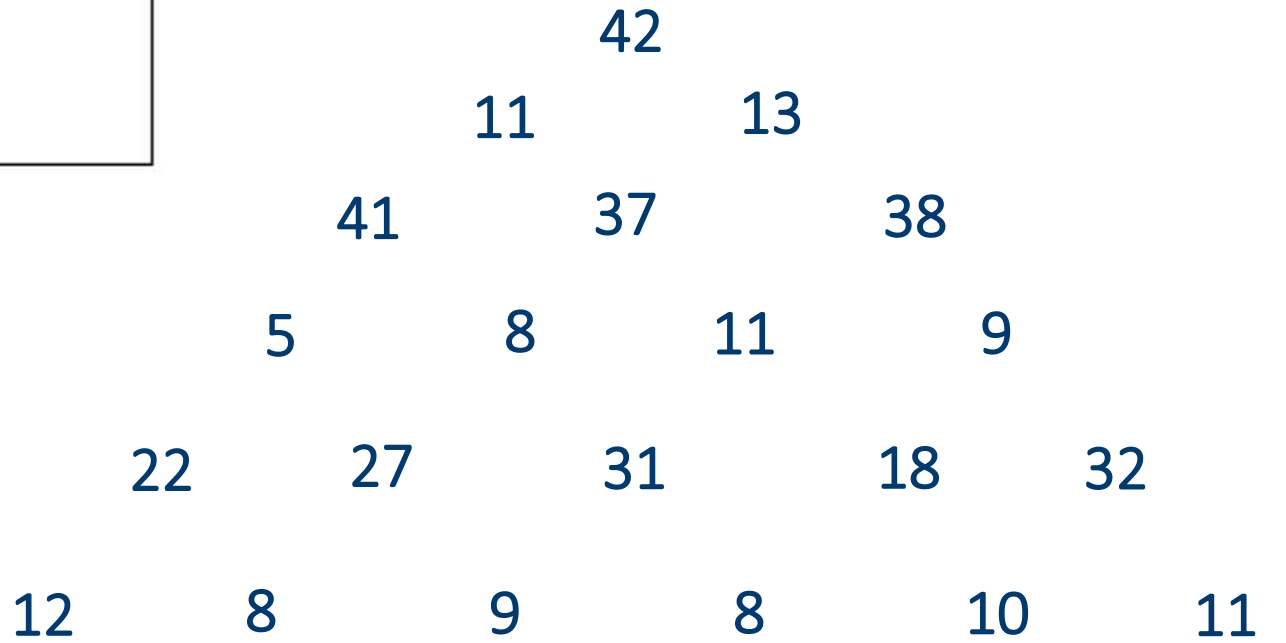
- $1 \leq A \leq 10$.
- Il valore di ciascun numero nella piramide è un intero positivo non superiore a 100.

Input e output: esempi

File input.txt	File output.txt
4 1 2 9 3 7 5 8 4 11 6	28
6 42 11 13 41 37 38 5 8 11 9 22 27 31 18 32 12 8 9 8 10 11	145

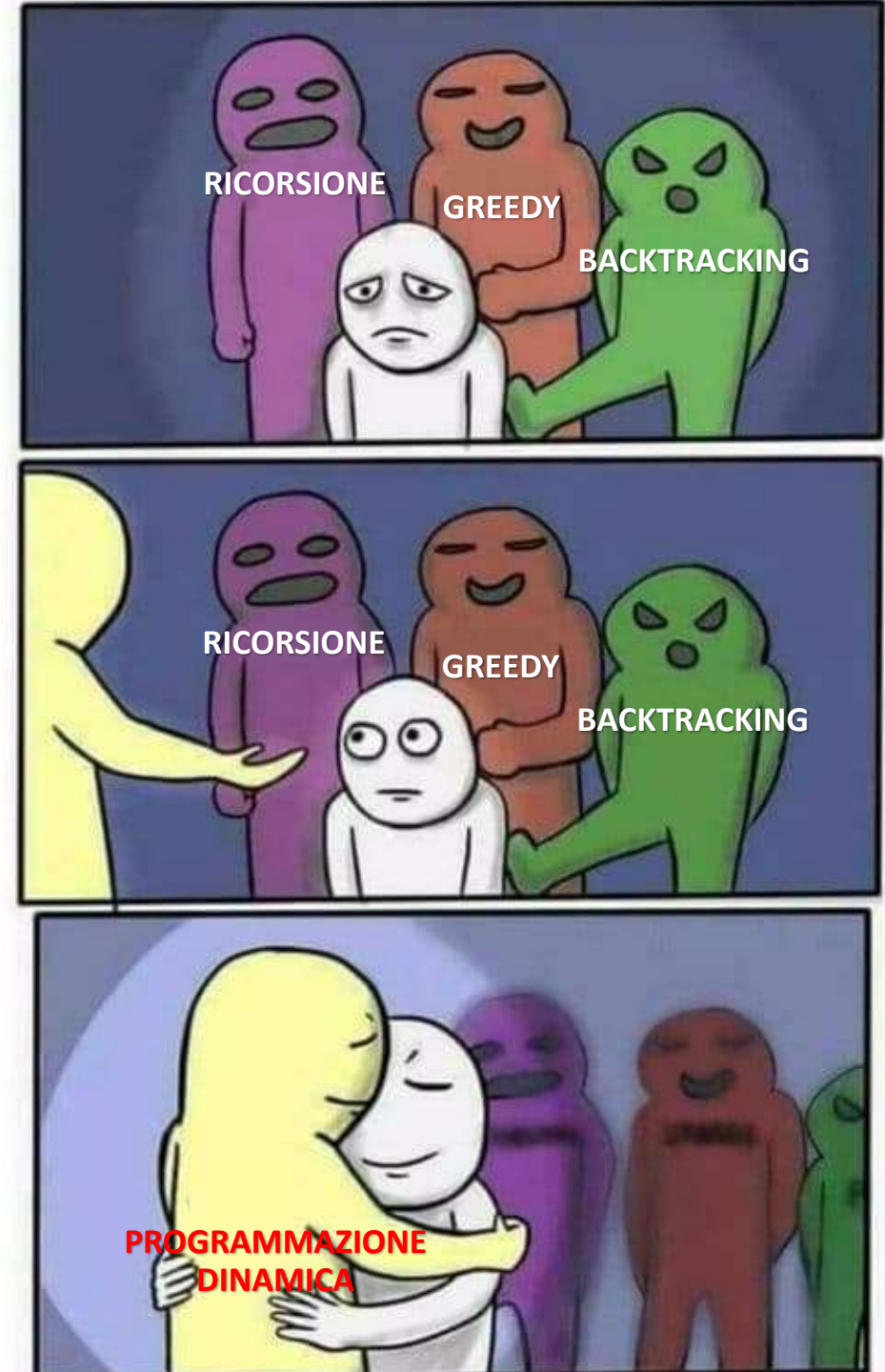
File input.txt	File output.txt
4 1 2 9 3 7 5 8 4 11 6	28

6	145
42	
11 13	
41 37 38	
5 8 11 9	
22 27 31 18 32	
12 8 9 8 10 11	



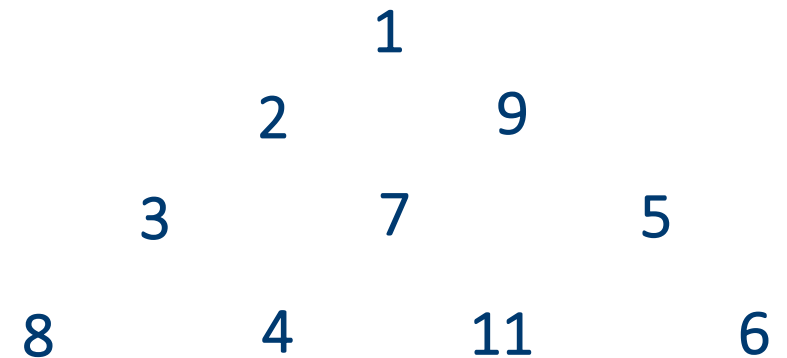
Riflessioni

- Approccio naturale: scendo verso il figlio più grande (greedy). Funziona?
- Soluzione ricorsiva "pura": prendo il massimo tra il valore della discesa massima a sinistra e il valore della discesa massima a destra.
 - È corretto?
 - È veloce?
 - Si può migliorare?

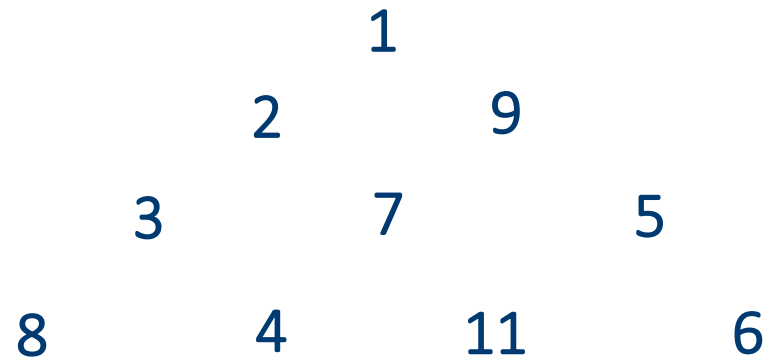


Riflessioni

- Approccio naturale: scendo verso il figlio più grande (greedy). Funziona?
- Soluzione ricorsiva "pura": prendo il massimo tra il valore della discesa massima a sinistra e il valore della discesa massima a destra.
 - È corretto?
 - È veloce?
 - Si può migliorare?
- Programmazione dinamica!



Riflessioni: Come memorizzare la piramide?



Riflessioni: Come memorizzare la piramide?

1
2 9
3 7 5
8 4 11 6

1			
2	9		
3	7	5	
8	4	11	6

Riflessioni: Sottoproblema?

1
2 9
3 7 5
8 4 11 6

1			
2	9		
3	7	5	
8	4	11	6

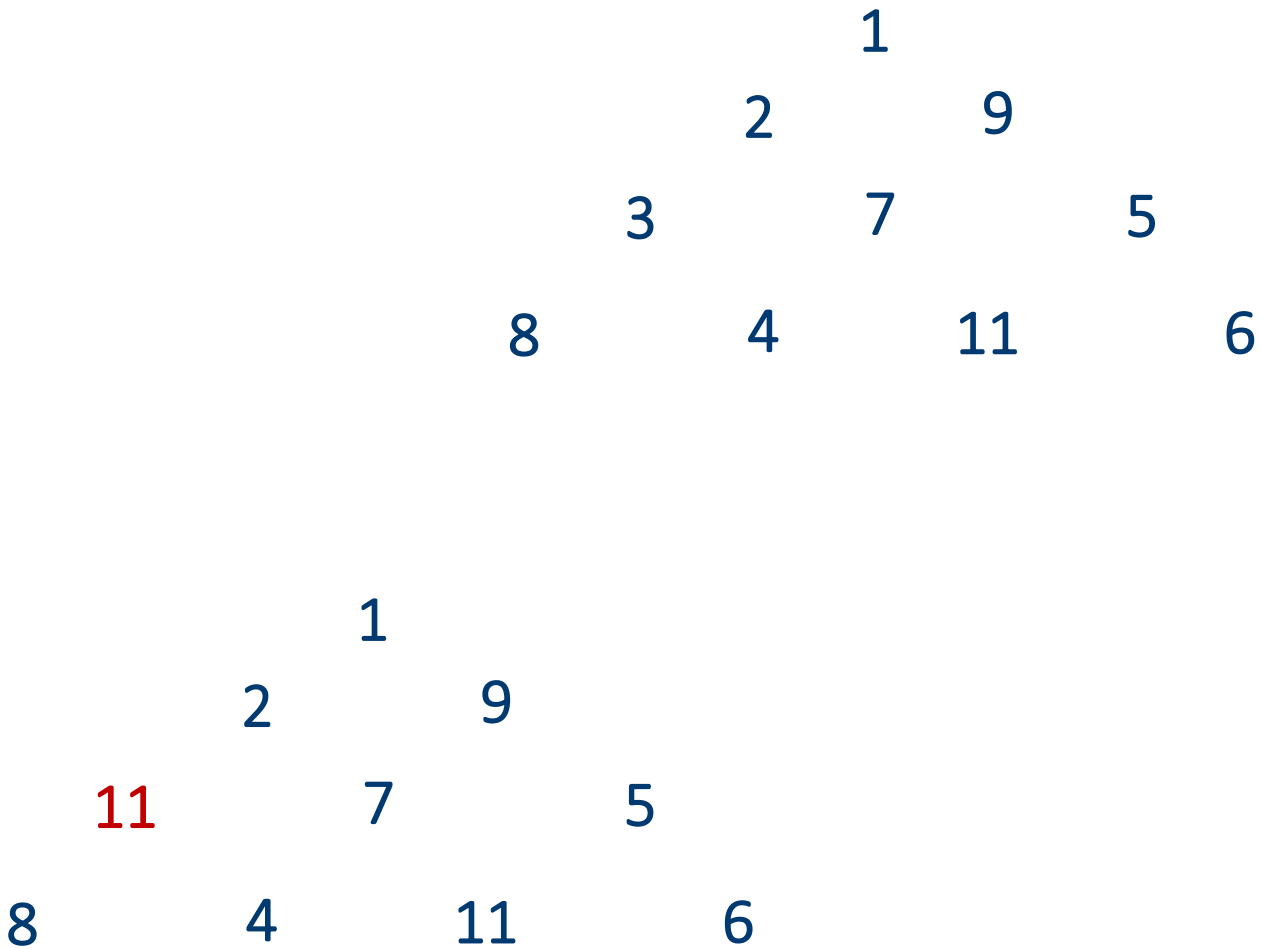
Riflessioni: Sottoproblema?

1
2 9
3 7 5
8 4 11 6

1			
2	9		
3	7	5	
8	4	11	6

1
2 9
3 7 5
8 4 11 6

Riflessioni

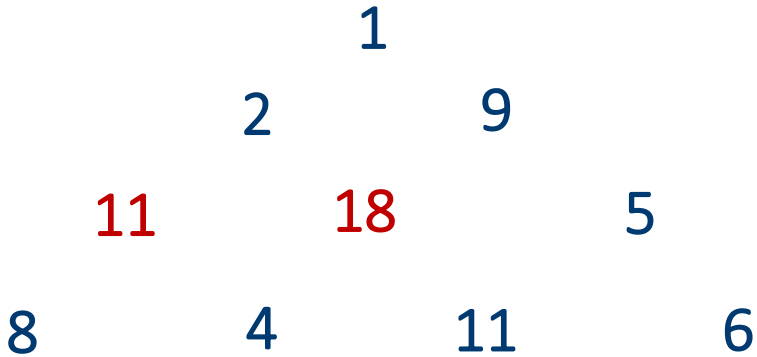


1			
2	9		
3	7	5	
8	4	11	6

Riflessioni



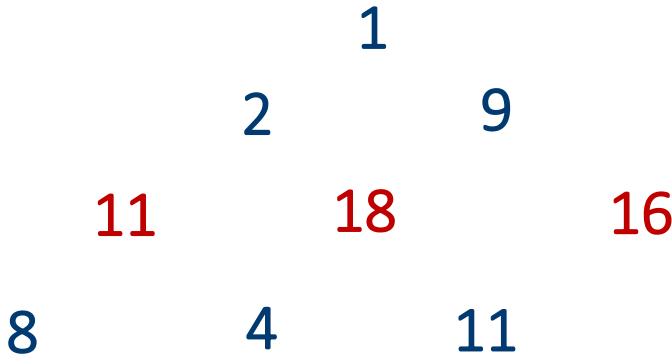
1			
2	9		
3	7	5	
8	4	11	6



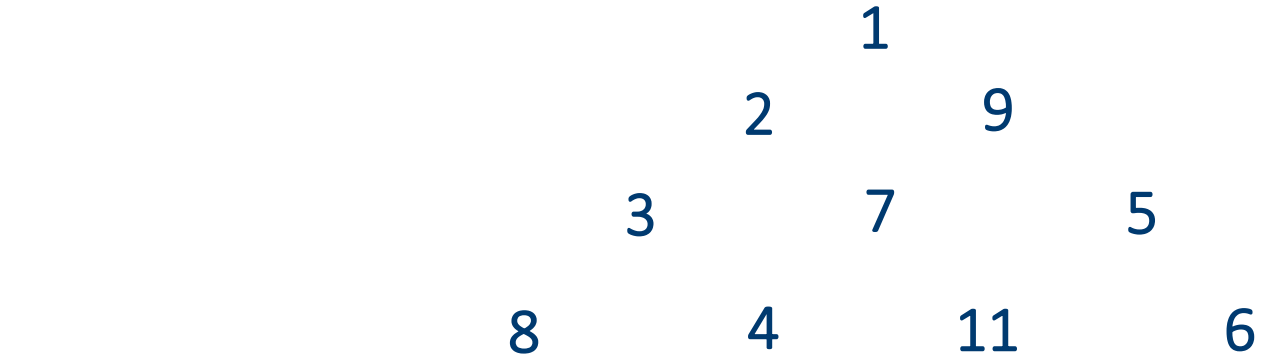
Riflessioni



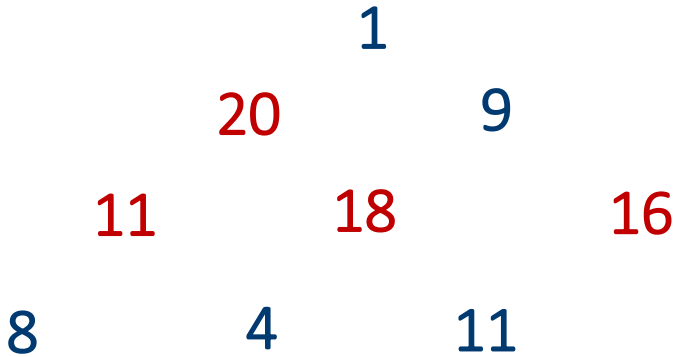
1			
2	9		
3	7	5	
8	4	11	6



Riflessioni



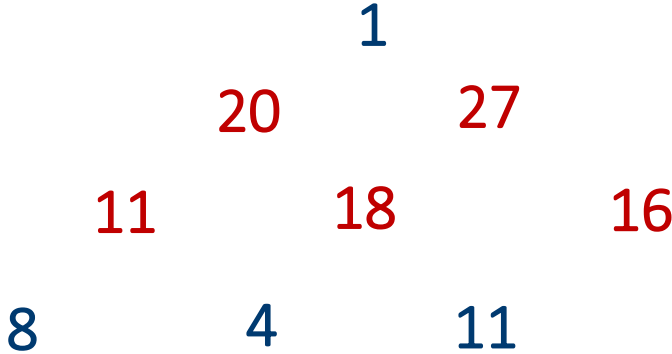
1			
2	9		
3	7	5	
8	4	11	6



Riflessioni



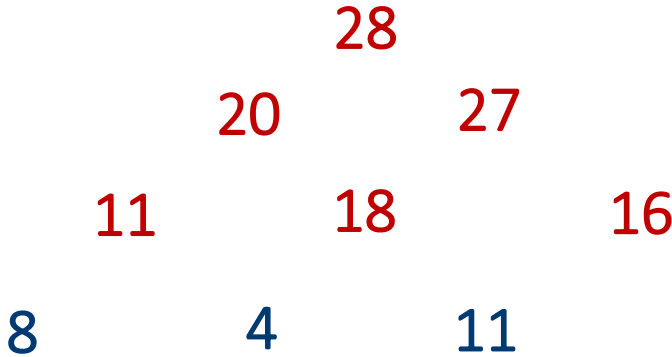
1			
2	9		
3	7	5	
8	4	11	6

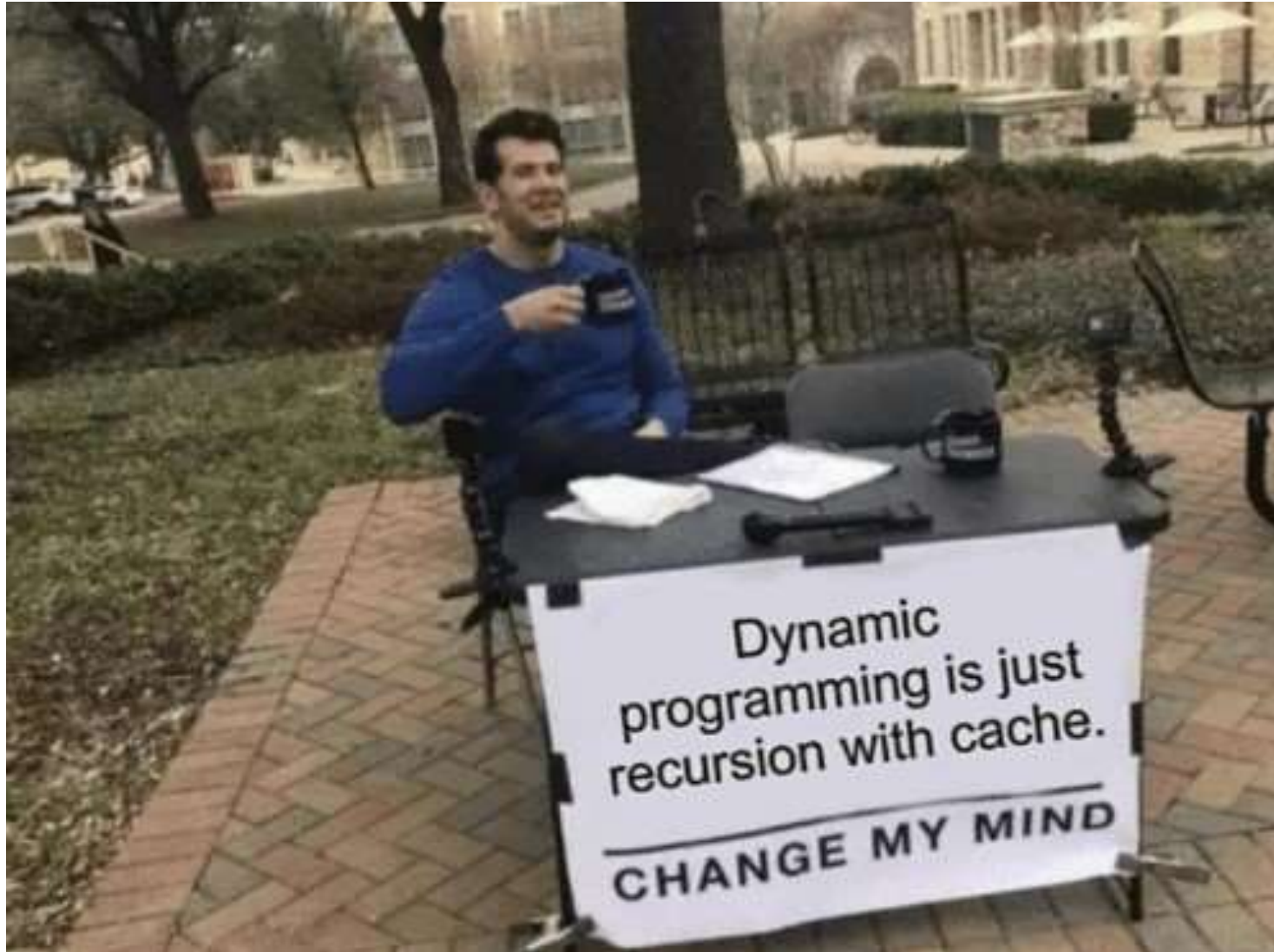


Riflessioni

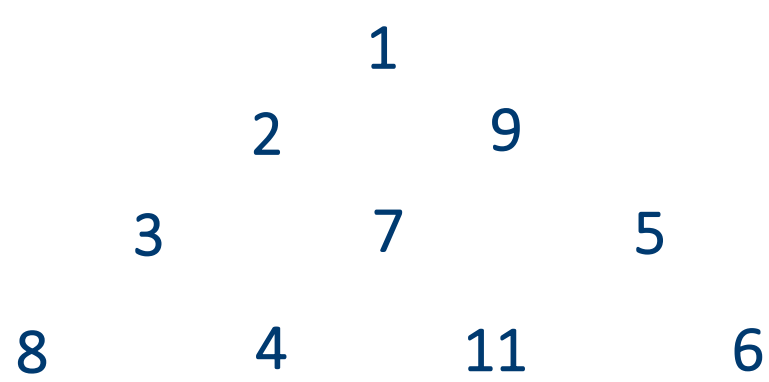


1			
2	9		
3	7	5	
8	4	11	6





Soluzione



1			
2	9		
3	7	5	
8	4	11	6

Un problema da LUISSMATICS (2018): Festa di laurea

https://training.olinfo.it/#/task/luiss_laurea/statement





Un problema dalle Pre-OII (2020): Sushi

https://training.olinfo.it/#/task/preoii_sushi/statement

Un problema dalle Pre-OII (2018): Treni

https://training.olinfo.it/#/task/preoii_treni/statement



Compiti a casa (Esercizi Olimpiadi)



Compiti a casa

Prova a risolvere i seguenti problemi:

- Missioni segrete (regionali 2008)
(dopo greedy, ricorsione: risolvo con programmazione dinamica!)
<https://training.olinfo.it/#/task/missioni/statement>
- Sushi variegato (pre OII 2017)
https://training.olinfo.it/#/task/preoii_yutaka/statement
- Esame di maturità (ABC 2016)
https://training.olinfo.it/#/task/abc_esame/statement
- Numeri di Figonacci (OIS 2014)
<https://training.olinfo.it/#/task/figonacci/statement>
- Minisupplì (LuissMatics 2018)
https://training.olinfo.it/#/task/luiss_suppli/statement